

Practica 5

```
1 //Four register with parameterized width in SystemVerilog and using
  interfaces
2
3 //Interface definition
4 interface FourReg_if #(parameter width = 8) ();
5     // shared signals
6     logic clk;
7     logic rst_n;
8     logic en_write;
9     logic [1:0] addr;
10    logic [width-1:0] write_data;
11    // register outputs
12    logic [width-1:0] out_0, out_1, out_2, out_3;
13
14    // modport for the DUT (module drives out_x and samples inputs)
15    modport DUT (
16        input  clk, rst_n, en_write, addr, write_data,
17        output out_0, out_1, out_2, out_3
18    );
19
20 endinterface
21
22 // DUT module definition
23 module FourReg (FourReg_if.DUT FR_if);
24
25     always_ff @(posedge FR_if.clk or negedge FR_if.rst_n) begin
26         if (!FR_if.rst_n) begin
27             FR_if.out_0 <= '0;
28             FR_if.out_1 <= '0;
29             FR_if.out_2 <= '0;
30             FR_if.out_3 <= '0;
31         end else if (FR_if.en_write) begin
32             case (FR_if.addr)
33                 2'b00: FR_if.out_0 <= FR_if.write_data;
34                 2'b01: FR_if.out_1 <= FR_if.write_data;
35                 2'b10: FR_if.out_2 <= FR_if.write_data;
```

```

36             2'b11: FR_if.out_3 <= FR_if.write_data;
37         endcase
38     end
39 end
40
41 endmodule
42
43 // Testbench module
44 module FourReg_tb;
45
46     FourReg_if #(8) FR_if(); // Instantiate the interface
47     FourReg dut(FR_if.DUT); // Instantiate the DUT
48
49     // Randomized test case class
50     class RandCase #(parameter width = 8);
51         rand bit en_write;
52         rand bit [1:0] addr;
53         rand bit [width-1:0] data;
54         function new(); endfunction
55     endclass
56
57     RandCase #(8) rc; // Random case object
58     integer total_cases = 0;
59     integer error_count = 0;
60     integer i;
61
62     //Verification process
63     initial begin

```

```

64
65 //Create waveform file
66 $dumpfile("FourReg_tb.vcd");
67 $dumpvars(0, FourReg_tb);
68
69 // Reset sequence
70 FR_if.rst_n = 0;
71 FR_if.clk = 0;
72 #10;
73 FR_if.rst_n = 1;
74 FR_if.clk = 1;
75 #10;
76
77 $display("Starting test sequence...");
78
79 rc = new();
80
81 for (i = 0; i < 8; i++) begin
82 void'(rc.randomize());
83 ///rc.randomize(); // Generate random values
84 total_cases++;
85
86 // Random data assignment to interface signals
87 FR_if.en_write = rc.en_write;
88 FR_if.addr = rc.addr;
89 FR_if.write_data = rc.data;
90
91 // Display current test case
92
91 // Display current test case
92 $display("---> DATA=%h / ADDR=%b / En_write=%b <---", rc.data,
rc.addr, rc.en_write);
93
94 // clk = 0 does not write
95 FR_if.clk = 0;
96 #10;
97 // clk = 1 triggers write if en_write is high
98 FR_if.clk = 1;
99 #10;
100
101 // Verification for written data
102 if (rc.en_write) begin
103 case (rc.addr)
104 2'b00: assert (FR_if.out_0 === rc.data)
105 else begin
106 $display("Error, data = %h wasn't written on addr = 00",
rc.data);
107 error_count++;
108 end
109 2'b01: assert (FR_if.out_1 === rc.data)
110 else begin
111 $display("Error, data = %h wasn't written on addr = 01",
rc.data);
112 error_count++;
113 end
114 2'b10: assert (FR_if.out_2 === rc.data)
115 else begin

```

```

116         $display("Error, data = %h wasn't written on addr = 10",
rc.data);
117         error_count++;
118     end
119     2'b11: assert (FR_if.out_3 === rc.data)
120     else begin
121         $display("Error, data = %h wasn't written on addr = 11",
rc.data);
122         error_count++;
123     end
124 endcase
125 end
126
127 if(rc.en_write == 0)begin
128     $display ("Enable to grite due to en_write = 0");
129     error_count++;
130 end
131
132 end
133
134 // Final Report
135 $display("---> Verification Resume <---");
136 $display("Reviewed cases: %0d", total_cases);
137 $display("Errors: %0d", error_count);
138 $display("Successes: %0d", total_cases - error_count);
139
140
141 end
142 endmodule
143
144

```

```

Log Share
[2025-11-02 00:15:23 UTC] xrun -Q -unbuffered '-timescale' '1ns/1ns' '-sysv' '-access' '+rw' '+ntb_random_seed_automatic' design.sv testbench.sv
TOOL: xrun 23.09-s001: Started on Nov 01, 2025 at 20:15:23 EDT
xrun: 23.09-s001: (c) Copyright 1995-2023 Cadence Design Systems, Inc.
Top level design units:
FourReg_tb
Loading snapshot worklib.FourReg_tb:sv ..... Done
SVSEED default: 1
xcelium> source /xcelium23.09/tools/xcelium/files/xmsimrc
xcelium> run
Starting test sequence...
---> DATA=17 / ADDR=00 / En_write=0 <---
Enable to grite due to en_write = 0
---> DATA=96 / ADDR=01 / En_write=0 <---
Enable to grite due to en_write = 0
---> DATA=97 / ADDR=01 / En_write=0 <---
Enable to grite due to en_write = 0
---> DATA=2c / ADDR=00 / En_write=0 <---
Enable to grite due to en_write = 0
---> DATA=46 / ADDR=00 / En_write=0 <---
Enable to grite due to en_write = 0
---> DATA=37 / ADDR=10 / En_write=0 <---
Enable to grite due to en_write = 0
---> DATA=4b / ADDR=01 / En_write=0 <---
Enable to grite due to en_write = 0
---> DATA=72 / ADDR=01 / En_write=1 <---
---> Verification Resume <---
Reviewed cases: 8
Errors: 7
Successes: 1
xmsim: *W,RNQUIE: Simulation is complete.
xcelium> exit
TOOL: xrun 23.09-s001: Exiting on Nov 01, 2025 at 20:15:24 EDT (total: 00:00:01)

```

//Four register with parameterized width in SystemVerilog and using interfaces

```

//Interface definition
interface FourReg_if #(parameter width = 8) ();
    // shared signals
    logic clk;
    logic rst_n;
    logic en_write;
    logic [1:0] addr;
    logic [width-1:0] write_data;
    // register outputs
    logic [width-1:0] out_0, out_1, out_2, out_3;

    // modport for the DUT (module drives out_x and samples inputs)
    modport DUT (
        input clk, rst_n, en_write, addr, write_data,
        output out_0, out_1, out_2, out_3
    );

endinterface

// DUT module definition
module FourReg (FourReg_if.DUT FR_if);

    always_ff @(posedge FR_if.clk or negedge FR_if.rst_n) begin
        if (!FR_if.rst_n) begin
            FR_if.out_0 <= '0;
            FR_if.out_1 <= '0;
            FR_if.out_2 <= '0;
            FR_if.out_3 <= '0;
        end else if (FR_if.en_write) begin
            case (FR_if.addr)
                2'b00: FR_if.out_0 <= FR_if.write_data;
                2'b01: FR_if.out_1 <= FR_if.write_data;
                2'b10: FR_if.out_2 <= FR_if.write_data;
                2'b11: FR_if.out_3 <= FR_if.write_data;
            endcase
        end
    end

endmodule

// Testbench module
module FourReg_tb;

```

```
FourReg_if #(8) FR_if(); // Instantiate the interface
FourReg dut(FR_if.DUT); // Instantiate the DUT
```

```
// Randomized test case class
class RandCase #(parameter width = 8);
    rand bit en_write;
    rand bit [1:0] addr;
    rand bit [width-1:0] data;
    function new(); endfunction
endclass
```

```
RandCase #(8) rc; // Random case object
integer total_cases = 0;
integer error_count = 0;
integer i;
```

```
//Verification process
initial begin
```

```
    //Create waveform file
    $dumpfile("FourReg_tb.vcd");
    $dumpvars(0, FourReg_tb);
```

```
    // Reset sequence
    FR_if.rst_n = 0;
    FR_if.clk = 0;
    #10;
    FR_if.rst_n = 1;
    FR_if.clk = 1;
    #10;
```

```
    $display("Starting test sequence...");
```

```
    rc = new();
```

```
for (i = 0; i < 8; i++) begin
    void'(rc.randomize());
    ///rc.randomize();    // Generate random values
    total_cases++;
```

```
    // Random data assignment to interface signals
    FR_if.en_write = rc.en_write;
    FR_if.addr = rc.addr;
    FR_if.write_data = rc.data;
```

```

// Display current test case
$display("----> DATA=%h / ADDR=%b / En_write=%b <---", rc.data, rc.addr, rc.en_write);

// clk = 0 does not write
FR_if.clk = 0;
#10;
// clk = 1 triggers write if en_write is high
FR_if.clk = 1;
#10;

// Verification for written data
if (rc.en_write) begin
case (rc.addr)
2'b00: assert (FR_if.out_0 === rc.data)
    else begin
        $display("Error, data = %h wasn't written on addr = 00", rc.data);
        error_count++;
    end
2'b01: assert (FR_if.out_1 === rc.data)
    else begin
        $display("Error, data = %h wasn't written on addr = 01", rc.data);
        error_count++;
    end
2'b10: assert (FR_if.out_2 === rc.data)
    else begin
        $display("Error, data = %h wasn't written on addr = 10", rc.data);
        error_count++;
    end
2'b11: assert (FR_if.out_3 === rc.data)
    else begin
        $display("Error, data = %h wasn't written on addr = 11", rc.data);
        error_count++;
    end
endcase
end

if(rc.en_write == 0)begin
    $display ("Enable to grite due to en_write = 0");
    error_count++;
end

end

```

```
// Final Report
$display("---> Verification Resume <---");
$display("Reviewed cases: %0d", total_cases);
$display("Errors: %0d", error_count);
$display("Successes: %0d", total_cases - error_count);

end
endmodule
```